

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

*I Lokesh S (192511037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Lokesh S

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Q1. Interviewer: “Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?”

Answer:

Query processing is the process of converting an SQL query into an efficient execution plan and producing the final result. It mainly involves **parsing, translation, optimization, execution, and result generation**. First, the DBMS parses the SQL query and checks its syntax and semantic correctness. The query is then translated into an internal representation such as relational algebra. During optimization, the optimizer generates different possible execution plans by considering **join order, indexes, access methods, selection and projection operations, and join algorithms**. It estimates the cost of each plan based on **disk I/O, CPU cost, memory usage, relation size, and intermediate results**. The optimizer compares these costs and selects the plan with the lowest estimated cost. Finally, the selected plan is executed by the query execution engine and the result is returned to the user.

Q2. Interviewer: “What is the difference between heuristic optimization and cost-based optimization? When would you use each?”

Answer:

Heuristic optimization improves a query using predefined rules rather than calculating the exact cost of every possible plan. Common rules include **pushing selections down, performing projections early, and joining smaller relations first**. It is fast and useful when a quick optimization is required. **Cost-based optimization**, on the other hand, generates multiple possible execution plans and estimates the cost of each plan using statistics such as table size, number of tuples, available indexes, disk I/O, CPU usage, and join methods. It then selects the plan with the lowest estimated cost. Heuristic optimization is suitable for **simple queries or fast optimization**, whereas cost-based optimization is preferred for **complex queries where selecting the most efficient execution plan is important**.

Q3. Interviewer: “How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.”

Answer:

In a real Java project, database connectivity can be implemented using **JDBC (Java Database Connectivity)**. First, the application establishes a connection using the database URL, username, and password. Then a Connection object is created, followed by a PreparedStatement or Statement for executing SQL queries. For parameterized queries,

PreparedStatement is preferred because it improves security and helps prevent SQL injection. The query is executed and the result is processed using a ResultSet. Finally, database resources are closed properly. **JDBC is designed specifically for Java applications**, while **ODBC (Open Database Connectivity)** is a general database connectivity standard that can be used by different programming languages through ODBC drivers. JDBC is commonly preferred in Java projects because it provides direct integration with Java applications.

Q4. Interviewer: “What are ACID properties? Give a real-world example where each property is critical.”

Answer:

ACID stands for **Atomicity, Consistency, Isolation, and Durability**. These properties ensure that database transactions are reliable and maintain data correctness. **Atomicity** means that a transaction is completed completely or not performed at all. For example, during a bank transfer, money should either be deducted from one account and credited to another, or neither operation should occur. **Consistency** ensures that a transaction changes the database from one valid state to another valid state. **Isolation** ensures that concurrent transactions do not interfere with each other or see incomplete changes. **Durability** means that once a transaction is committed, its changes remain stored even if a system failure occurs. These properties are especially important in **banking, payment systems, airline reservations, and e-commerce applications**.

Q5. Interviewer: “Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?”

Answer:

Concurrency problems occur when multiple transactions access the same data simultaneously. A **dirty read** occurs when one transaction reads data written by another transaction that has not yet committed. A **lost update** occurs when one transaction's update is overwritten by another transaction. An **unrepeatable read** occurs when a transaction reads the same data twice but obtains different values because another transaction modified the data between the reads. **Two-Phase Locking (2PL)** controls concurrent access using locks. During the **growing phase**, a transaction acquires the required locks, and during the **shrinking phase**, it releases them and cannot acquire new locks. By controlling conflicting read and write operations, 2PL ensures **conflict serializability** and prevents transactions from incorrectly interfering with each other.

Q6. Interviewer: “What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.”

Answer:

A **deadlock** occurs when two or more transactions wait indefinitely for resources or locks held by each other. For example, T1 holds a lock on A and waits for B, while T2 holds B and waits for A. Deadlock can be detected using a **Wait-For Graph (WFG)**. In this graph, each transaction is represented as a node, and an edge $T1 \rightarrow T2$ means T1 is waiting for a resource held by T2. If the graph contains a **cycle**, a deadlock exists. Two deadlock prevention techniques are **Wait-Die** and **Wound-Wait**. In Wait-Die, an older transaction is allowed to wait, while a younger transaction requesting a lock held by an older transaction is aborted. In Wound-Wait, an older transaction can abort a younger transaction holding the required lock, while a younger transaction waits for an older transaction.

Q7. Interviewer: “Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?”

Answer:

In **Immediate Update**, changes made by a transaction can be written to the database before the transaction commits. Therefore, after a system failure, recovery may require both **UNDO and REDO** operations. Committed transactions are REDONE, while uncommitted transactions are UNDONE using the log. In **Deferred Update**, changes are not written to the database until the transaction commits. Therefore, uncommitted changes do not need to be undone, and recovery mainly requires **REDO of committed transactions**. Immediate Update provides flexibility when database changes need to be written during transaction execution, but recovery is more complex. Deferred Update is simpler and is preferred when updates can safely be delayed until commit.

Q8. Interviewer: “Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?”

Answer:

Shadow Paging is a database recovery technique that maintains two page tables: a **shadow page table** and a **current page table**. The shadow page table points to the stable pages of the database, while the current page table is used for modifications made during a transaction. When a page is modified, a new copy of the page is created instead of overwriting the original

page. The current page table is updated to point to the new page. If the transaction commits, the current page table becomes permanent. If a system failure occurs, the database can discard the current page table and use the shadow page table to restore the previous consistent state. Its advantages are **simple recovery and no traditional UNDO/REDO processing**. Its limitations include additional storage and page-management overhead, fragmentation, and reduced efficiency when many pages are modified.

Q9. Interviewer: “What is a serializability schedule? How do you test for conflict serializability using a precedence graph?”

Answer:

A **serializable schedule** is a concurrent execution of transactions whose final result is equivalent to some serial execution of those transactions. To test **conflict serializability**, we construct a **precedence graph**. Each transaction is represented as a node. An edge $T_i \rightarrow T_j$ is added when an operation of T_i conflicts with an operation of T_j on the same data item and T_i 's operation occurs first. Conflicts occur between **read-write**, **write-read**, and **write-write** operations. After constructing the graph, we check for cycles. If the graph is **acyclic**, the schedule is conflict-serializable, and a topological ordering gives the equivalent serial order. If a cycle exists, the schedule is **not conflict-serializable**.

Q10. Interviewer: “Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?”

Answer:

Basic 2PL divides transaction execution into two phases: the **growing phase**, where locks are acquired, and the **shrinking phase**, where locks are released. Once a transaction releases a lock, it cannot acquire another lock. **Strict 2PL** keeps all **exclusive locks** until the transaction commits or aborts. This prevents dirty reads and cascading rollbacks and makes recovery easier. **Rigorous 2PL** is stricter because it keeps **both shared and exclusive locks** until commit or abort. All three protocols provide conflict serializability, but they differ in when locks are released. **Strict 2PL is widely preferred in practical database systems** because it provides strong isolation, prevents cascading rollback, simplifies recovery, and offers a good balance between correctness and performance.